

BigK API Guide

For: Mobile, web, and admin-panel developers working with the BigK Wallet and BigK Admin backend services. **Version:** 1.1

1. Product Overview

BigK Wallet is a comprehensive reward platform application designed for managing KCC Coins. Key capabilities include Wallet & Transfers, Rewards & XP, K-Mall Commerce, Achievements & Social.

BigK Admin is the operational backend and admin API surface for platform staff. It covers wallet administration, ledger visibility, treasury operations, reward management, analytics, app settings, support moderation, and audit-log review.

2. Token Legal & Product Model

- **Internal Utility Only:** KCC Coins operate strictly as a closed-loop internal reward unit within the BigK ecosystem.
 - **Not a Deposit Account:** The wallet balance is not a bank deposit and KCC Coins are not publicly tradable.
 - **No Cash Out:** Coins cannot be withdrawn back to fiat currency unless explicitly permitted.
 - **Ecosystem Use:** Transacting is limited exclusively to approved platform flows.
-

3. Architecture & Service Boundaries

BigK Wallet uses a decoupled multi-service architecture: - **Primary Identity & Tokens:** kccid (Central Authentication Service). - **Wallet-Domain API:** BigK Admin Backend (/api/v1). - **Commerce API:** PlenorHub Backend.

3.1 Backend Ownership Boundaries

Domain	Owning Backend	Notes
Identity, login, OAuth/PKCE, 2FA, token claims, global RBAC context	KCC ID	KCC ID is the source of identity truth and issues tokens consumed by BigK and approved platform clients.
Wallet profiles, balances, transfers, KCC ledger,	BigK Admin Backend	Sensitive wallet and ledger actions must be

Domain	Owning Backend	Notes
rewards, achievements, BigK admin APIs		implemented or reviewed by the BigK backend owner.
Merchant catalog, merchant products, order fulfillment, merchant portal, payout workflows	PlenorHub Backend	PlenorHub owns merchant commerce data and syncs approved merchant events to BigK where needed.

3.2 Admin Endpoint Ownership Rule

New admin endpoints that expose ledger data, affect balances, approve merchants, moderate users/content, change permissions, or expose audit logs must be owned or reviewed by the relevant service owner. Every such endpoint must include RBAC, audit logging, validation, and abuse controls before release.

4. Identity & Token Contract

JWT Validation Contract

- **Issuer:** `https://auth.bigkpay.com`
- **Audience Requirement:** Tokens must include the `bigk_wallet` audience.
- **Required Claims:** `sub`, `iss`, `aud`, `exp`.
- **Supported Optional Claims:** `email`, `name`, `preferred_username`, `picture`.

Token Versioning & 2FA

- **Token Versioning:** KCC ID tracks global token versions per user. If a user's password changes or they are forcefully logged out, their token version increments. Tokens with older versions will be rejected with `401 Unauthorized` and the client must force a logout.
- **2FA Session Continuity:** Enabling or disabling TOTP does not invalidate existing BigK Wallet sessions. The current session remains active and the client should show a friendly confirmation message. A future login may still require TOTP verification depending on the client and policy.
- **2FA Enforcement:** BigK Wallet enforces Two-Factor Authentication. During login or registration, the backend may return `2fa_enrollment_required` or `2fa_required` errors. The client must handle these exceptions and transition to the TOTP setup or verification screens before completing authentication.

5. Wallet Provisioning Lifecycle

- **Idempotency:** `POST /wallet/provision` is strictly idempotent. Repeated calls return the existing wallet.
- **1 KCC ID user = 1 BigK Wallet:** The system relies on KCC ID as the source of truth for identity mapping.

6. Global API Conventions

- **Base URL:** `https://api.bigkpay.com/api`
 - **Pagination:** Cursor-based formatting (`?per_page=20&page=1`). Responses include a meta object.
 - **Error Format:** Unified `{"message": "...", "errors": {"field": ["..."]}}`
 - **Idempotency:** Mutation requests (like `/app/transfer`) should include the Idempotency-Key header.
-

7. Funding, Stripe & Exchange Rates

- **Authoritative Rate Locking:** GET `/payments/exchange-rate` locks rates; others are purely informative.
 - **Wallet Credit Timing:** The KCC wallet is strictly credited upon explicit asynchronous Stripe webhook confirmation. Client-side success redirects alone do not guarantee credit.
-

8. Endpoint Reference

8.1 Authentication & SSO

POST /wallet/provision

Registers or retrieves wallet. - **Response:**

```
{
  "wallet": {
    "id": 12,
    "balance": "150.50"
  },
  "message": "Success",
  "created": false
}
```

GET /wallet/me

Check linked wallet. - **Response:**

```
{
  "wallet": {
    "id": 12,
    "balance": "150.50"
  }
}
```

POST /wallet/link-sso

Link SSO. - **Request:**

```
{  
  "sso_token": "..."  
}
```

• **Response:**

```
{  
  "message": "SSO linked"  
}
```

8.2 Sessions (Auth)

GET /auth/sessions

Get sessions. - **Response:**

```
{  
  "data": [  
    {  
      "id": "sess_123",  
      "device": "iPhone"  
    }  
  ]  
}
```

DELETE /auth/sessions/{id}

Revoke session. - **Response:**

```
{  
  "message": "Revoked"  
}
```

DELETE /auth/sessions

Revoke all. - **Response:**

```
{  
  "message": "All revoked"  
}
```

POST /auth/Logout

Logout. - **Response:**

```
{  
  "message": "Logged out"  
}
```

8.3 Core Profile & Settings

GET /profile

Get profile. - **Response:**

```
{
  "data": {
    "id": 12,
    "handle": "johndoe"
  }
}
```

PUT /profile

Update profile. - **Request:**

```
{
  "display_name": "John"
}
```

- **Response:**

```
{
  "message": "Updated"
}
```

POST /profile/avatar

Upload avatar. - **Response:**

```
{
  "message": "Updated",
  "avatar_url": "..."
}
```

POST /profile/device-token

Register push token. - **Request:**

```
{
  "token": "fcm_..."
}
```

- **Response:**

```
{
  "message": "Registered"
}
```

GET /profile/{id}

View public profile. - **Response:**

```
{
  "data": {
    "id": 45,
    "handle": "jane"
  }
}
```

PUT /profile/currency

Update currency. - **Request:**

```
{
  "currency": "EUR"
}
```

- **Response:**

```
{
  "message": "Updated"
}
```

GET /profile/notification-settings

Get notif settings. - **Response:**

```
{
  "data": {
    "push_enabled": true
  }
}
```

PUT /profile/notifications

Update notif settings. - **Request:**

```
{
  "push_enabled": true
}
```

- **Response:**

```
{
  "message": "Updated"
}
```

PUT /profile/privacy

Update privacy. - **Request:**

```
{
  "is_public": false
}
```

- **Response:**

```
{
  "message": "Updated"
}
```

GET /app/limits

Get limits. - **Response:**

```
{
  "data": {
    "daily_transfer_limit": "50000"
  }
}
```

PUT /app/limits

Set limits. - **Request:**

```
{
  "daily_limit": "20000"
}
```

- **Response:**

```
{
  "message": "Updated"
}
```

8.4 Balances, Transfers & Contacts

GET /app/me

Get core balances. - **Response:**

```
{
  "data": {
    "balance": "150.50",
    "total_xp": 1200
  }
}
```

GET /app/transactions

Get history. - **Response:**

```
{
  "data": [
    {
      "id": 845,
      "amount": "25.00",
      "type": "transfer",
      "is_credit": true
    }
  ],
}
```

```
    "meta": {
      "current_page": 1
    }
  }
```

POST /app/transfer

P2P transfer (Requires Idempotency-Key). - **Request:**

```
{
  "to_handle": "jane",
  "amount": "15.00"
}
```

- **Response:**

```
{
  "data": {
    "id": 846,
    "status": "completed"
  }
}
```

GET /app/contacts

Get contacts. - **Response:**

```
{
  "data": [
    {
      "id": 45,
      "handle": "jane"
    }
  ]
}
```

GET /app/saved-wallets

List saved wallets. - **Response:**

```
{
  "data": [
    {
      "nickname": "Mom"
    }
  ]
}
```

POST /app/saved-wallets

Save wallet. - **Request:**


```
{
  "wallet_id": 45
}
```

- **Response:**

```
{
  "message": "Saved"
}
```

PUT /app/saved-wallets/{id}

Update saved wallet. - **Request:**

```
{
  "nickname": "Mom"
}
```

- **Response:**

```
{
  "message": "Updated"
}
```

DELETE /app/saved-wallets/{id}

Delete saved wallet. - **Response:**

```
{
  "message": "Deleted"
}
```

8.5 Rewards, Tasks & XP

GET /app/tasks

Get tasks. - **Response:**

```
{
  "data": [
    {
      "id": "tsk_1",
      "reward_amount": "5.00"
    }
  ]
}
```

POST /app/tasks/{task}/claim

Claim task. - **Response:**

```
{
  "message": "Earned 5.00 KCC!",
  "data": {
    "new_balance": "155.50"
  }
}
```

```
}  
}
```

POST /app/tasks/{task}/submit

Submit proof. - **Response:**

```
{  
  "message": "Submitted"  
}
```

GET /app/campaigns/active

Get active promotional campaigns for the logged-in wallet user. Campaigns group reward tasks into seasonal or feature-exploration missions. - **Response:**

```
{  
  "data": [  
    {  
      "id": 1,  
      "external_id": "0b4e9a8b-6d7a-4df4-80f5-3a1e9f4b3c20",  
      "title": "Explore BigK Launch Week",  
      "subtitle": "Complete missions and earn KCC",  
      "type": "seasonal",  
      "status": "live",  
      "starts_at": "2026-05-09T00:00:00+00:00",  
      "ends_at": "2026-05-16T23:59:59+00:00",  
      "banner_gradient": ["#2563EB", "#14B8A6"],  
      "cta_label": "Explore & Earn",  
      "cta_route": "/rewards",  
      "tasks_count": 3,  
      "tasks": [  
        {  
          "id": 12,  
          "title": "Make your first transfer",  
          "task_type": "first_transfer",  
          "reward_amount": "20.00",  
          "can_claim": false,  
          "completion_count": 0  
        }  
      ]  
    }  
  ]  
}
```

GET /app/campaigns/{campaign}

Get one active promotional campaign and its mission tasks. - **Response:** Same campaign object as GET /app/campaigns/active.

Admin Promotional Campaign APIs

Admin users manage seasonal and feature-exploration campaigns from the Rewards → Campaigns tab.

Method	Endpoint	Purpose
GET	/rewards/campaigns	List campaigns
POST	/rewards/campaigns	Create a campaign
GET	/rewards/campaigns/{campaign}	View campaign details
PUT	/rewards/campaigns/{campaign}	Update campaign
DELETE	/rewards/campaigns/{campaign}	Delete campaign
POST	/rewards/campaigns/{campaign}/clone	Clone campaign as draft
GET	/rewards/campaigns/{campaign}/audience-preview	Preview eligible wallets for campaign targeting
POST	/rewards/campaigns/{campaign}/notify	Send wallet inbox and/or email notifications to eligible wallets
POST	/rewards/campaigns/{campaign}/tasks/{task}	Attach an existing reward task as a campaign mission
DELETE	/rewards/campaigns/{campaign}/tasks/{task}	Detach a campaign mission

Campaign fields include title, subtitle, description, type, status, starts_at, ends_at, banner_gradient, cta_label, cta_route, targeting, and metadata.

The targeting object supports: - audience: all, new_users, or specific_wallets - wallet_ids: wallet IDs for specific_wallets - new_user_days: signup window for new_users - min_mall_purchases, max_mall_purchases, mall_purchase_days - min_transactions, max_transactions, transaction_days - min_balance, max_balance - min_age, max_age - created_after, created_before - notification_channels: inbox, email

Frontend behavior: - The admin frontend exposes a Campaigns tab under Rewards for creating campaigns, configuring targeting, previewing audiences, notifying eligible users, and attaching existing tasks. - The wallet home screen fetches GET /app/campaigns/active and displays active campaigns only when they are live/scheduled, in date range, and the wallet matches targeting rules.

GET /app/achievements

Get achievements. - **Response:**

```
{
  "data": [
    {
      "title": "First Transfer"
    }
  ]
}
```

GET /xp/balance

Get XP. - **Response:**

```
{
  "data": {
    "balance": 400
  }
}
```

POST /xp/redeem

Redeem XP (Requires Idempotency-Key). - **Request:**

```
{
  "amount": 100
}
```

- **Response:**

```
{
  "message": "Redeemed",
  "kcc_credited": "1.00"
}
```

8.6 Social, Inbox & Global

GET /friends

Get friends. - **Response:**

```
{
  "data": []
}
```

POST /friends/request

Send friend request. - **Request:**

```
{
  "wallet_id": 45
}
```

- **Response:**

```
{
  "message": "Sent"
}
```

POST /app/support-submissions

Submit user feedback, feature ideas, or support requests. - **Request:**

```
{
  "type": "feedback",
  "subject": "App suggestion",
  "message": "It would be great to have...",
  "journey": "Profile",
  "rating": 5,
  "contact_email": "user@example.com"
}
```

- **Response:**

```
{
  "message": "Thanks – your feedback has been sent."
}
```

GET /app/inbox

Get inbox messages. - **Response:**

```
{
  "data": [
    {
      "title": "Welcome!"
    }
  ]
}
```

POST /payments/stripe/checkout

Create checkout session. - **Request:**

```
{
  "amount_kcc": 1000
}
```

- **Response:**

```
{
  "data": {
    "checkout_url": "..."
  }
}
```

GET /currencies

Get currencies. - **Response:**

```
{
  "data": [
    {
      "code": "USD"
    }
  ]
}
```

9. Admin API Governance

Admin APIs are sensitive because they can expose ledger data, operational metrics, user information, merchant workflows, payout information, and moderation actions. All admin modules must follow these rules:

- Admin requests must use KCC ID-issued JWTs.
- Admin routes must run behind `kcc.id` plus role middleware.
- The current BigK admin route group allows `super_admin` and `plenorhub_admin`.
- Balance-affecting or moderation actions must be auditable.
- Read-heavy reporting endpoints should support pagination, filtering, and date ranges.
- Sensitive backend logic should not be duplicated across services unless the ownership boundary has been explicitly approved.

9.1 Admin RBAC Model

BigK uses KCC ID as the identity and role source. The BigK backend resolves the role from the KCC ID token and applies route middleware.

Current role hierarchy:

Role	Effective Access
<code>super_admin</code>	Full platform access across BigK admin APIs.
<code>plenorhub_admin</code>	PlenorHub/admin operational access where allowed by BigK route middleware.
<code>merchant_admin</code>	Merchant-owned management access for assigned merchant IDs.
<code>merchant_staff</code>	Limited merchant staff access for assigned merchant IDs.
<code>app</code>	Regular wallet app user access.

The hierarchy is implemented so higher roles inherit access to lower role groups where explicitly allowed. Route-level role checks remain the final authority.

Recommended permission model for future admin expansion:

Permission Area	Example Permissions
Wallets	wallets.read, wallets.update, wallets.lock, wallets.reset_password
Ledger	ledger.read, ledger.export, ledger.adjust, ledger.reconcile
Treasury	treasury.read, treasury.mint, treasury.burn, treasury.reward, treasury.snapshot
Rewards	rewards.read, rewards.write, campaigns.write, task_submissions.review
Merchants	merchants.read, merchants.approve, merchants.reject, merchant_payouts.review
Moderation	support.read, support.update, users.invite, users.delete
Reports	reports.read, reports.custom, analytics.read
Audit	audit.read, audit.export
Settings	settings.read, settings.write

When adding endpoints, prefer explicit permissions over broad role checks for high-risk actions.

9.2 Module Ownership

Module	Backend Owner	Frontend Owner	Notes
Wallet users, balances, transfers, saved wallets	BigK	BigK Admin	BigK owns wallet ledger integrity.
All transactions / ledger visibility	BigK	BigK Admin	Ledger access is sensitive and should remain under BigK-controlled admin permissions.
Treasury mint, burn, reward, supply snapshots	BigK	BigK Admin only unless approved	Requires strict audit logs and operational review.
Rewards, tasks, achievements, campaigns	BigK	BigK Admin	Manual task review is a moderation/admin operation.

Module	Backend Owner	Frontend Owner	Notes
Merchant catalog, products, orders, merchant application approval	PlenorHub	PlenorHub Merchant/Admin	BigK may receive sync events for wallet-side attribution.
Merchant payout approval	PlenorHub/BigK depending on settlement flow	Admin UI	Final owner should be confirmed before implementation.
KCC ID users, clients, roles, 2FA policy	KCC ID	KCC ID dashboard/admin	Identity and authorization remain centralized.

9.3 Backend Implementation Policy

BigK-controlled backend services should remain the implementation authority for:

- wallet balance changes;
- ledger adjustments or transaction overrides;
- merchant approval/rejection workflows;
- payout approval or settlement actions;
- user deletion, suspension, lock/unlock, or password reset;
- RBAC, token, 2FA, or KCC ID client policy changes;
- audit-log modification or privileged audit-log access.

Any endpoint in these areas must be implemented by, or reviewed and merged by, the owning BigK/KCC backend team.

10. Admin API Reference

All endpoints in this section are under:

`https://api.bigkpay.com/api/v1`

Unless stated otherwise, admin endpoints require:

Authorization: Bearer <kcc_id_admin_access_token>

Accept: application/json

Content-Type: application/json

Current BigK admin route protection:

`kcc.id + kcc.role:super_admin,plenorhub_admin`

10.1 Admin Authentication

Method	Endpoint	Purpose
GET	/admin/profile	Get current admin profile and resolved permissions.
POST	/admin/logout	Revoke current admin session.
POST	/admin/change-password	Change current admin password.
POST	/auth/accept-invitation	Accept a user or merchant invitation.

10.2 Ledger & Transactions

Method	Endpoint	Purpose
GET	/ledger/transactions	List ledger transactions across wallets.

Expected filters should include date range, wallet ID, transaction type, direction, reference, status, and pagination where supported.

Ledger records may include source wallet, destination wallet, amount, transaction type, reference, timestamp, and `performed_by_admin_id` for admin-originated actions.

10.3 Wallet Administration

Method	Endpoint	Purpose
GET	/wallets	List wallets.
POST	/wallets	Create/admin-provision a wallet.
GET	/wallets/{wallet}	View wallet details.
PUT	/wallets/{wallet}	Update wallet metadata/status.
DELETE	/wallets/{wallet}	Delete/deactivate wallet according to backend policy.
POST	/wallets/{wallet}/unlock	Unlock a wallet.
POST	/wallets/{wallet}/reset-password	Admin password reset for a wallet user.

Wallet admin actions should be logged because they affect account access and user state.

10.4 Treasury & Sensitive Balance Operations

Method	Endpoint	Purpose
GET	/treasury/supply	View supply summary.
GET	/treasury/inflation	View inflation report.

Method	Endpoint	Purpose
GET	/treasury/economics	View economics overview.
POST	/treasury/recalculate	Recalculate treasury totals.
POST	/treasury/snapshot	Create treasury snapshot.
POST	/treasury/mint	Mint KCC to a wallet.
POST	/treasury/burn	Burn KCC from a wallet.
POST	/treasury/reward	Grant reward KCC to a wallet.

mint, burn, and reward are high-risk endpoints. They should require a reason/reference, idempotency where appropriate, and audit logging.

10.5 Payments & Refunds

Method	Endpoint	Purpose
GET	/payments/intents	List payment intents.
GET	/payments/intents/{intent}	View a payment intent.
POST	/payments/intents/{intent}/sync	Sync status from payment provider.
DELETE	/payments/intents/{intent}	Cancel a payment intent.
GET	/payments/refunds	List refunds.
GET	/payments/refunds/stats	Refund statistics.
POST	/payments/refunds	Create refund request.
POST	/payments/refunds/{id}/process	Process refund.
POST	/payments/refunds/{id}/cancel	Cancel refund.

10.6 Rewards, Tasks, Campaigns & Achievements

Method	Endpoint	Purpose
GET	/rewards/tasks	List reward tasks.
POST	/rewards/tasks	Create reward task.
PUT	/rewards/tasks/{task}	Update reward task.
DELETE	/rewards/tasks/{task}	Delete reward task.
GET	/rewards/tasks/{task}/completions	View task completions.

Method	Endpoint	Purpose
POST	/rewards/tasks/{task}/clone	Clone reward task.
GET	/rewards/campaigns	List campaigns.
POST	/rewards/campaigns	Create campaign.
GET	/rewards/campaigns/{campaign}	View campaign.
PUT	/rewards/campaigns/{campaign}	Update campaign.
DELETE	/rewards/campaigns/{campaign}	Delete campaign.
POST	/rewards/campaigns/{campaign}/clone	Clone campaign.
GET	/rewards/campaigns/{campaign}/audience-preview	Preview eligible wallets.
POST	/rewards/campaigns/{campaign}/notify	Notify eligible wallets.
POST	/rewards/campaigns/{campaign}/tasks/{task}	Attach task to campaign.
DELETE	/rewards/campaigns/{campaign}/tasks/{task}	Detach task from campaign.
GET	/rewards/achievements	List achievements.
GET	/rewards/achievements/stats	Achievement stats.
POST	/rewards/achievements	Create achievement.
PUT	/rewards/achievements/{id}	Update achievement.
DELETE	/rewards/achievements/{id}	Delete achievement.
POST	/rewards/achievements/{id}/toggle	Enable/disable achievement.

10.7 Moderation, Requests & Support

Method	Endpoint	Purpose
GET	/admin/task-submissions	List submitted reward-task proofs.
GET	/admin/task-submissions/stats	Task-submission moderation stats.
GET	/admin/task-submissions/{id}	View submitted proof.
POST	/admin/task-submissions/{id}/approve	Approve submitted proof.

Method	Endpoint	Purpose
POST	/admin/task-submissions/{id}/reject	Reject submitted proof.
POST	/admin/task-submissions/bulk-approve	Bulk approve submitted proofs.
GET	/admin/support-submissions	List feedback/support submissions.
GET	/admin/support-submissions/stats	Support submission stats.
POST	/admin/support-submissions/{id}/status	Update support submission status.
DELETE	/admin/support-submissions/{id}	Delete support submission.

Moderation actions should capture reviewer identity, status, notes/reason, and timestamps.

10.8 Analytics, Reports & Dashboard Statistics

Method	Endpoint	Purpose
GET	/analytics/overview	Dashboard overview metrics.
GET	/analytics/transactions/daily	Daily transaction trends.
GET	/analytics/wallets/growth	Wallet growth trends.
GET	/analytics/payments/success-rate	Payment success metrics.
GET	/analytics/transactions/top	Top transactions.
GET	/analytics/revenue	Revenue metrics.
GET	/analytics/behavior	User behavior analytics.
GET	/analytics/merchants/{merchantId}	Merchant-specific analytics.
POST	/analytics/reports/custom	Custom report generation.

Expected report filters: from, to, currency, merchant ID, wallet ID, transaction type, status, grouping, and pagination/export mode where supported.

10.9 Alerts, Settings & Management Operations

Method	Endpoint	Purpose
GET	/alerts	List platform alerts.
POST	/alerts	Create alert.
GET	/alerts/{alert}	View alert.

Method	Endpoint	Purpose
PUT	/alerts/{alert}	Update alert.
DELETE	/alerts/{alert}	Delete alert.
GET	/admin/settings	List app settings.
PUT	/admin/settings/{key}	Update one setting.
POST	/admin/settings/bulk	Bulk update settings.
POST	/admin/settings/seed	Seed default settings.

10.10 Users, Invitations & Account Management

Method	Endpoint	Purpose
POST	/admin/users/invite	Invite admin/user account.
DELETE	/admin/users/{userId}	Delete account.
DELETE	/admin/users/{userId}/cancel-invitation	Cancel user invitation.
POST	/admin/merchants/invite	Invite merchant account.
GET	/admin/invitations	List invitations.
DELETE	/admin/invitations/{invitationId}	Cancel invitation.
POST	/admin/invitations/{invitationId}/resend	Resend invitation.

10.11 Merchant Payout Administration

Method	Endpoint	Purpose
GET	/admin/payouts	List merchant payout requests.
GET	/admin/payouts/stats	Payout stats.
POST	/admin/payouts/{id}/approve	Approve payout.
POST	/admin/payouts/{id}/reject	Reject payout.
GET	/admin/merchants/{merchantId}/payment-details	View merchant payout/payment details.

10.12 Audit Logs

Method	Endpoint	Purpose
GET	/audit-logs	List audit logs.
GET	/audit-logs/{auditLog}	View one audit log entry.

Audit logs should be treated as read-only operational evidence. They should not be modified by frontend applications.

Document Owner: Munene